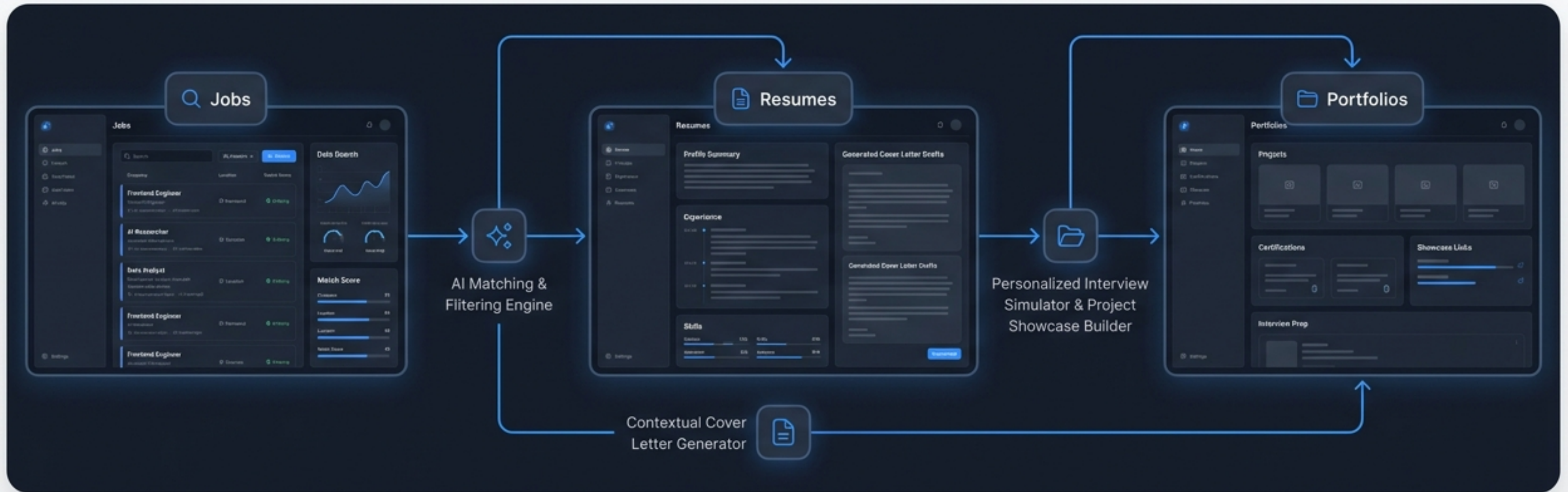


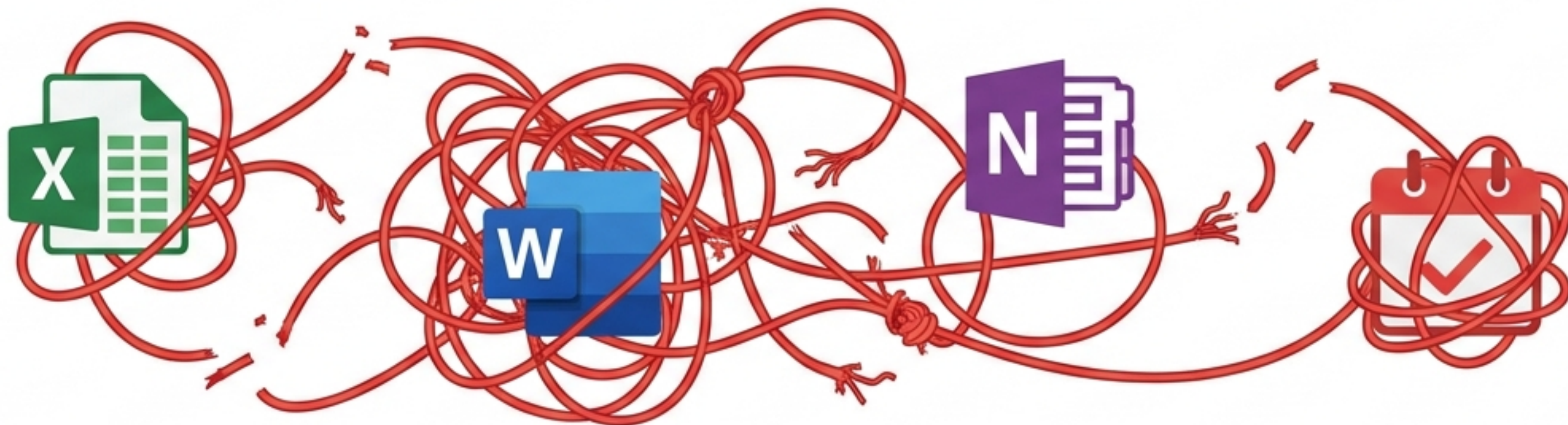
JobFilter: 공고 탐색을 넘어 취업 준비를 연결하다

컴퓨터정보과·인공지능소프트웨어과 학생을 위한 맞춤 취업 공고 필터링 및 End-to-End 취업 준비 플랫폼.

기능을 무작정 늘리는 것이 아닌, 공고 발견부터 자소서, 면접, 포트폴리오까지 하나의 유기적인 흐름으로 연결하는 아키텍처 검증 보고서.



파편화된 취업 준비의 한계



섞인 엑셀 파일

학교 공고에 개발·사무·영업 등
무관한 전공 혼재

적합성 판단 불가

나에게 맞는 공고인지
스스로 파악하기 어려움

분산된 자료

공고별 자소서와 준비물이
여러 플랫폼에 흩어짐

피드백 부재

자소서에서 부족한 부분을
객관적으로 찾기 어려움

일정 누락

마감일, 면접일, 팔로업
일정 추적의 어려움

약한 연결고리

공고, 자소서, 면접,
포트폴리오 간의 데이터 단절

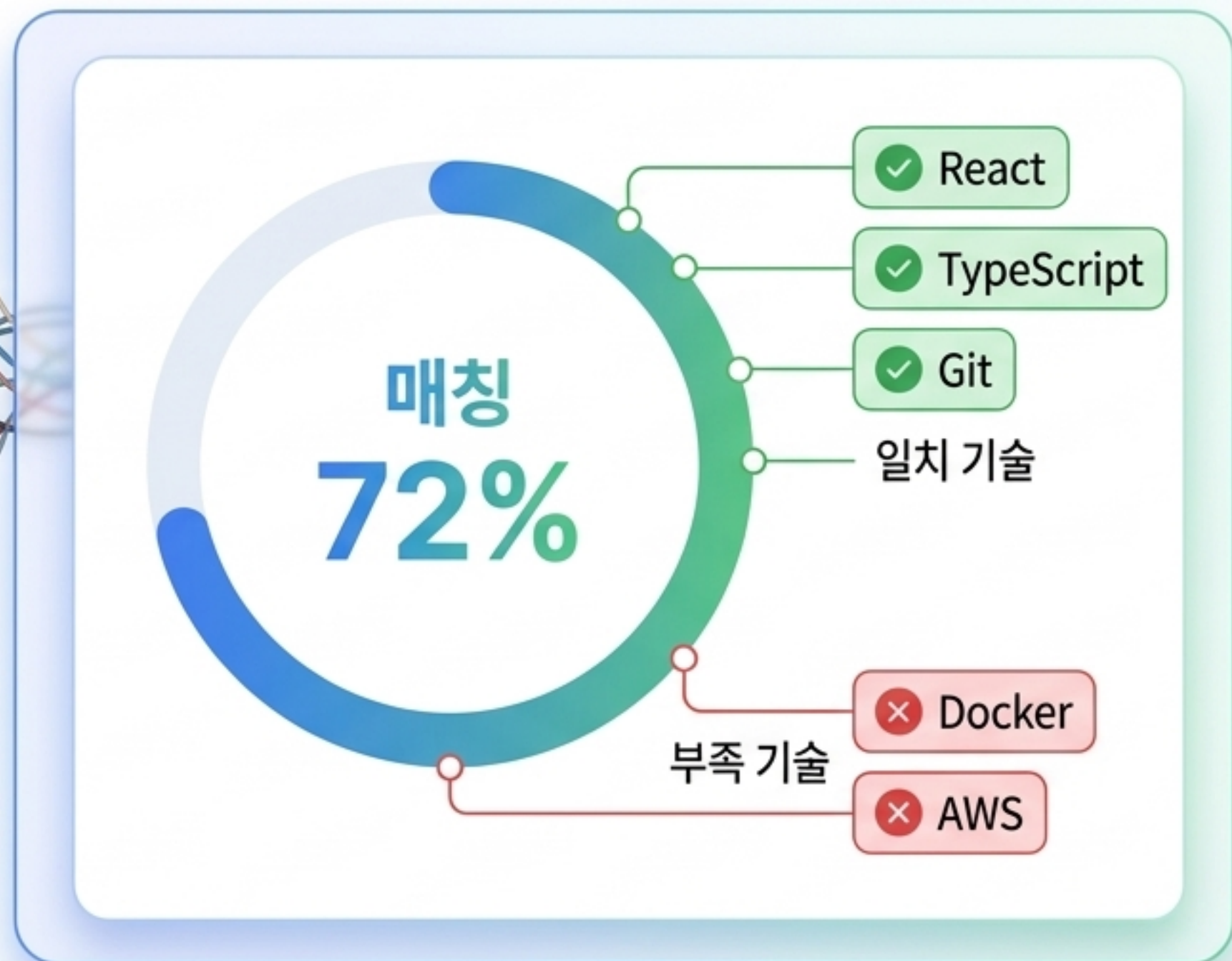
하나의 파이프라인으로 연결된 서비스 흐름



JobFilter 서비스 구성 매트릭스

학생 기능 (Student)	관리자 기능 (Admin)	리크루터 및 향후 (Recruiter & Future)
맞춤 채용광고 ✓ 현재 구현	Excel/CSV 공고 업로드 ✓ 현재 구현	리크루터 가입 ✓ 현재 구현
지원 현황 테이블·칸반 ✓ 현재 구현	업로드 미리보기·dry-run ✓ 현재 구현	관리자 승인 대기 ✓ 현재 구현
지원 건 Command Center ✓ 현재 구현	공고 일괄 처리 ✓ 현재 구현	기업 공고 등록·수정·삭제 ✓ 현재 구현
지원 준비율 ● 1차 구현	사용자 관리 ✓ 현재 구현	관리자 검토 후 공고 노출 ✓ 현재 구현
자소서 룰 기반 코칭 ✓ 현재 구현	질문·공지 관리 ✓ 현재 구현	리크루터 학생 탐색 ● 향후 계획
피드백 문장 하이라이트 ✓ 현재 구현	통계·취업 기록 ● 1차 구현	공개 동의 세분화 ● 향후 계획
JD 키워드 히트맵 ✓ 현재 구현		지원 활동 분석 ● 향후 계획
자소서 중복 검사·버전 diff ✓ 현재 구현		
경험 저장소 ✓ 현재 구현		
면접 질문·답변·모의 면접 ● 1차 구현		
포트폴리오·GitHub 분석 ● 1차 구현		
로드맵·캘린더·알림 ● 1차 구현		

설명 가능한 매칭 점수 (Explainable Job Matching)



```
Backend File Paths

1 lib/matching.ts
2 api/listings/route.ts
3 api/dashboard/priority-jobs/route.ts
```

검색 및 필터 조건

회사명, 직무, 지역, 마감 임박(7일 이내), 신입/경력 조건 결합.

설명 가능한 로직 (Explainable Matching)

- 숫자 하나만 던져주는 블랙박스 방식을 탈피.
- 공통 matching 모듈을 통해 목록 화면과 대시보드가 동일한 계산 기준 공유.
- 기술 일치, 직무 유사도, 경력 조건, 마감 긴급도를 종합하여 '적합 근거' 반환.

💡 Insight: 단순한 합격 예측이 아닌, 기술 일치/직무/경력/마감 긴급도를 종합한 '참고 지표' 제공

지원 건 통합 커맨드 센터 (Command Center 1차)



- 분리되어 있던 화면들을 하나의 지원 건(Job) 데이터 중심으로 통합.
- 뷰어 전환: 전체 흐름을 보는 테이블 뷰 ↔ 진행 단계를 직관화하는 칸반 뷰 전환 지원.
- 컨텍스트 유지 (Context Passing): 자소서 화면으로 이동 시 JobId, 회사명, 직무명을 파라미터로 전달하여 맥락 단절 방지.
- 추적 기능: 상태(준비 중→지원 완료→서류 합격→면접 예정→최종 합격) 변경 이력 및 담당자 연락처 영구 추적.

💡 개발 주안점: 현재 면접/포트폴리오는 화면 이동 중심이며, 지원 건 데이터로 직접 집계하는 기능은 향후 P1 과제로 분리.

지원 준비율과 대시보드 구조화

UX (사용자 경험)



지원 준비율 산정 기준 (4요소 25%씩):

대시보드 오늘의 할 일

- ☒ 1. 자소서 연결
- ☒ 2. 체크리스트 완료
- ☒ 3. 면접 일정
- ☐ 4. 팔로업 일정

현재 준비율: 75%

Backend Architecture (백엔드 아키텍처)

☐ 기존 (Before): GET /api / jobs
-> 알림 강제 생성 (부작용 및 비용 증가)

☒ 개선 (After): GET /api / jobs (조회만) &
POST /api/notifications/sync (명시적 동기화)

알림 아키텍처 개선: 단순 조회 시 알림이 생성되는
부작용 해결. 조회와 동기화를 분리하여 아키텍처
안정화.

마감/면접/팔로업 기반으로 긴급도를 구분하여
별도의 /today 페이지 없이 대시보드에 통합 렌더링.

비용과 투명성을 잡은 '룰 기반' 자소서 코칭

✓ 11가지 정밀 분석 항목

- ✓ 1. 분량 확인
- ✓ 2. 공허한 표현(클리셰)
- ✓ 3. 수치 데이터 포함 여부
- ✓ 4. STAR 구조 확인
- ✓ 5. 직군 맞춤 키워드
- ✓ 6. 지나친 장문
- ✓ 7. 험업 표현 유무
- ✓ 8. 첫 문장 패턴
- ⚠ 9. 접속사 남용
- ⚠ 10. 수동태 사용
- ✓ 11. 추상적 표현

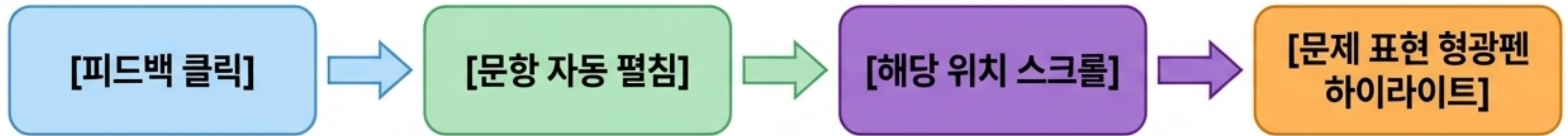
💡 왜 AI를 쓰지 않았나?

전략적 선택:

- **지속 가능성:** 학생 대상 무료 서비스 유지를 위해 토큰 비용(API) 영구 제거 및 1차 점검 로직 자체 구현.
- **설명 가능성:** 블랙박스 AI가 아닌, 학생과 시스템이 명확한 기준을 공유. 단순한 합격 판정기가 아닌 스스로 개선할 수 있는 '작성 보조 도구'로 포지셔닝.

자소서 피드백 하이라이트 & 버전 Diff

피드백 UX 개선: 분석 결과를 읽고 문장을 다시 찾는 불편함 제거.
정규식 역추적을 통해 분석이 '실제 편집 행동'으로 즉각 이어짐.

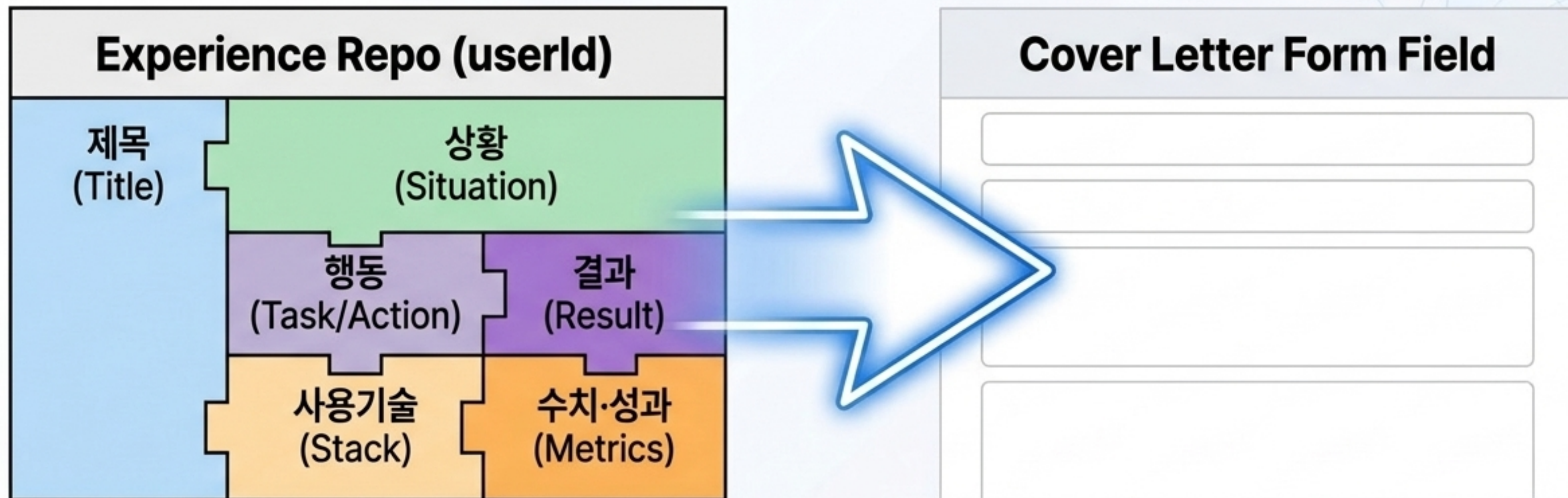


자소서 버전 Diff 및 자기표절 검사:

- 문항별 변경 내용을 **적색**(이전) / **녹색**(변경)으로 직관적 시각화.
- 점수 변화와 실제 문장 변화를 동시 트래킹. 중복 및 자기표절 검사 기능 통합.

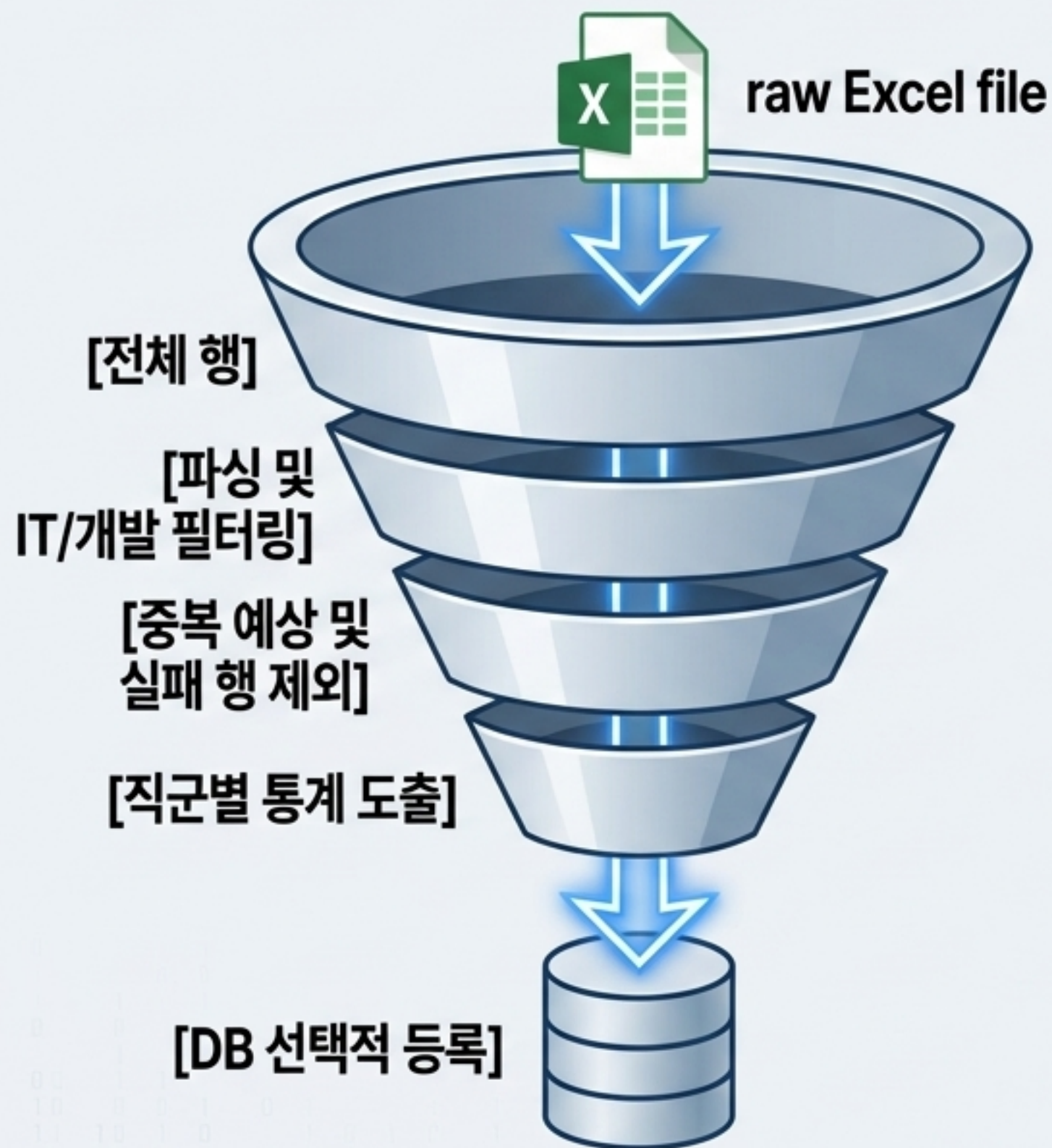
The screenshot shows a diff tool interface with a dark theme. At the top, it says 'v1' followed by a color-coded bar and 'codediff'. On the right, there are tabs for 'horgy' and a menu icon. The main area displays a comparison of two text versions. The left side shows the original text with line numbers 1 through 5. The right side shows the modified text with line numbers 1 through 5. A specific change is highlighted: on the left, a line is marked with a red minus sign and the text is in red: '- 자는 이 회사에 큰 기여를 할 수 있습니다.'. On the right, the corresponding line is marked with a green plus sign and the text is in green: '+ 자는 이 회사의 A 프로젝트 경험을 바탕으로 B 분야에서 구체적인 성과를 낼 수 있습니다.'. The text is wrapped to fit the width of the diff view.

STAR 기반 경험 저장소 (Experience Repo)



- ✓ **STAR 기반 아키텍처:** 비슷한 프로젝트 경험을 매번 새로 작성하는 페인포인트 해결. 상황/행동/결과/기술/성과를 구조화된 블록 형태로 userId에 종속시켜 저장.
- ✓ **데이터 재사용성 극대화:** AI가 지어내는(Hallucination) 것이 아님. 학생이 기록한 '실제 본인 데이터'를 자소서 문항의 첫 번째 슬롯에 즉각 삽입(Insert)하여 뼈대로 활용.

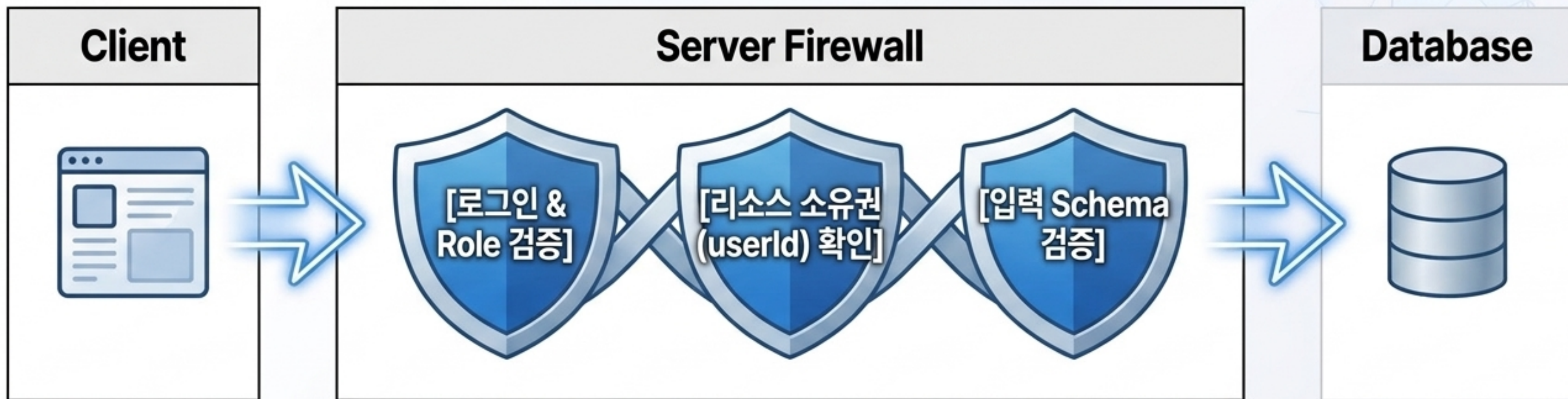
관리자: 데이터 품질을 위한 'Dry-run' 업로드



데이터 품질을 위한 'Dry-run' 프로세스

- 사전 검증 샌드박스: 형식이 일정하지 않은 엑셀(무관 전공, 날짜 오류 혼재)의 직접 삽입을 차단. 서버 DB 저장 전, 메모리 상에서 파싱/필터 결과를 미리보기 화면으로 출력.
- 안정성 장치: 파일 크기 및 최대 행(Row) 수 제한 로직 적용. 관리자는 중복/파싱 실패 행을 눈으로 확인한 뒤, 검증된 타겟 행만 '선택적 등록'하여 운영 데이터 오염 원천 차단.

리크루터 및 API 보안·데이터 설계



리크루터 워크플로우

가입 ➡ isApproved 서버 확인
➡ 기업 공고 등록 ➡ 관리자 검토
➡ 학생 노출 (미승인 시 비활성)

API 보안 및 방어선 구축

클라이언트 UI 숨김에 의존하지 않음.
6대 입력 검증 Schema 강제:

1. URL 규칙
2. 날짜 포맷
3. 이메일
5. 태그 수
4. 문자열 길이
6. 비밀번호 복잡도

데이터 설계 현황

- 현재 JSON 문자열 필드 API 변환 및 Prisma 마이그레이션 공통화 진행 중.
- 공개 API 방어를 위한 메모리 기반 Rate Limit 적용 (향후 Vercel 다중 인스턴스 대응을 위해 Redis/Upstash 전환 필요성 명시).

상태 점검: 테스트 결과 및 현재 구현의 한계 명확화

[완료 및 통과]

- ✓ 매칭 로직 검증
- ✓ 리크루터 권한 방어
- ✓ API 보안 테스트
- ✓ 핵심 단위 테스트 6개 통과

```
✓ prisma generate  
✓ tsc --noEmit  
✓ npm run lint  
✓ npm run test
```

[부분 완료 (현재의 한계)]

솔직한 리포트:

- API 통합: 단위 테스트는 완료되었으나, 실제 API Route 기반 소유권 통합 테스트 잔여.
- 데이터 �핑: 포트폴리오/면접 모듈의 화면은 오픈되었으나, 지원 건과 데이터베이스 차원의 '직접 연결'은 부분 미완료.
- DB 마이그레이션: Experience 스키마의 로컬 생성을 넘어 운영 DB 실제 적용 및 안정성 확인 필요.

다음 개발 우선순위 (Roadmap)

개발 원칙: 새로운 기능 추가보다 권한, 알림, 업로드 안정성 등 기반 확보가 최우선.

[P0: 기반 및 안정성] (최우선 과제)

- | | |
|----------------------|-----------------------------------|
| 1. 실제 API 소유권 통합 테스트 | 2. Experience 운영 DB 마이그레이션 적용 확인 |
| 3. 전체 API DTO 정리 | 4. 알림 구조화 5. 업로드 실패 행·원자성 보강 |

[P1: 데이터 연결 고도화] (다음 단계)

- | | | | |
|----------------------------|------------------|-------------|----------|
| 1. 포트폴리오·면접 답변의 지원 건 직접 연결 | 2. 로드맵과 공고 기술 연결 | | |
| 3. 기술 증거 매핑 | 4. 회사·공고 조사 노트 | 5. 지원 활동 분석 | 6. 면접 회고 |

[Hold: 부가 기능 보류] (기본 흐름 완성 후 검토)

음성 AI 면접 / AI 자소서 자동 채어 자동 생성 / 커뮤니티 기능 / 무제한 크롤링

취업 준비의 End-to-End 플랫폼: 최종 기대 효과

학생 (Student)

- 나에게 맞는 공고의 빠른 발견,
- 일정 누락 감소, 흩어진 경험 데이터의 완벽한 구조화 및 재사용.

학교·관리자 (School & Admin)

- 엑셀 공고 데이터의 원천적인 품질 향상,
- IT·개발 직무 자동 필터링,
- 향후 익명 통계 기반 취업 흐름 파악.

기업·리크루터 (Recruiter)

- 승인 기반의 안전한 공고 등록 환경 구축,
- 향후 공개 동의 학생 탐색을 위한 데이터 파이프라인 확보.

서비스 (Service Platform)

- 무분별한 외부 AI API 의존을 탈피하여 비용을 절감하고, 사용자 본인의 데이터를 재사용하는 학과 맞춤형 생태계 구축.

최종 메시지: JobFilter의 핵심은 단순 페이지 수를 늘리는 것이 아닙니다.
공고 발견부터 준비 완료까지, 분절된 과정을 하나의 완벽히 연결된 커맨드 센터로 통합하는 것입니다.